

Analisi tecnica del notebook di previsione del fabbisogno energetico

Maurizio Mormone
Autore dello Studio

OpenAI Codex/Prism
Analisi tecnica, revisione e validazione

9 aprile 2026

Abstract

Il presente documento sintetizza l'attività di revisione tecnica e metodologica svolta su un notebook dedicato alla previsione del fabbisogno energetico orario di un prosumer. Il lavoro nasce dall'esigenza di riesaminare criticamente una prima implementazione del modello, inizialmente funzionante sul piano operativo ma non ancora pienamente solida sotto il profilo metodologico, soprattutto nella gestione della normalizzazione dei target e nella corretta interpretazione delle metriche di valutazione. L'intervento di revisione ha consentito di ricostruire in modo rigoroso il flusso della pipeline, chiarire la separazione tra training, validation e test, verificare il comportamento del preprocessing e ricondurre la valutazione finale alla scala fisica originale delle grandezze previste.

Il risultato di questa attività non è stato un semplice affinamento formale del codice, ma un passaggio sostanziale da una prima versione del modello a una versione metodologicamente corretta, più leggibile, più verificabile e soprattutto più affidabile nell'interpretazione delle prestazioni. In questo senso, la revisione ha permesso di trasformare un prototipo utile a fini esplorativi in una base di lavoro molto più coerente per successive evoluzioni verso scenari applicativi reali.

Premessa

Il documento si colloca all'interno di un percorso di studio e sviluppo dedicato al forecasting dei consumi energetici, maturato a partire da un approfondimento teorico sul problema della previsione multi-step di serie temporali energetiche e sul possibile impiego di modelli neurali ricorrenti in presenza di variabili esogene meteorologiche. Su questa base è stato sviluppato un primo prototipo operativo, pensato per tradurre in pratica le ipotesi teoriche e verificare la fattibilità di una pipeline end-to-end capace di stimare consumo, produzione fotovoltaica e bilancio netto su orizzonte settimanale.

Successivamente, tale prototipo è stato sottoposto a una prima revisione con il supporto di OpenAI Codex, che ha condotto alla versione denominata **Previsione Fabbisogno v01**. Pur rappresentando un avanzamento importante in termini di struttura e organizzazione del notebook, questa versione non ha restituito risultati pienamente soddisfacenti sul piano interpretativo e metodologico. Per questo motivo è stata richiesta un'ulteriore analisi tramite Prism, con l'obiettivo di individuare in modo più preciso le criticità residue della pipeline, verificare la correttezza delle scelte implementative e proporre miglioramenti in grado di rendere il modello più robusto, più coerente e più credibile in vista di applicazioni future.

1 Descrizione del programma

Il notebook `prism-uploads/Previsione Fabbisogno v05_refactored_final v03_2.ipynb` implementa una pipeline completa di previsione del fabbisogno energetico orario per un prosumer, cioè un soggetto che consuma energia elettrica e, allo stesso tempo, produce energia tramite un impianto fotovoltaico. L'obiettivo operativo è stimare, per ciascuna ora dei sette giorni successivi, il consumo atteso, la produzione fotovoltaica prevista e il bilancio netto risultante, così da determinare sia l'energia da acquistare dalla rete sia l'eventuale eccedenza da immettere.

Dal punto di vista strutturale, il notebook segue un flusso logico chiaro e abbastanza coerente. Si parte dalla definizione dei parametri principali, che fissano uno storico di sei mesi con granularità oraria, una finestra passata di 168 ore e un orizzonte di previsione di altre 168 ore. In pratica, il modello osserva una settimana di storia e genera una previsione oraria sull'intera settimana successiva. Questa scelta è ragionevole in ottica dimostrativa, perché permette di catturare pattern giornalieri e settimanali senza introdurre una complessità eccessiva.

La prima parte operativa del notebook è dedicata alla generazione dei dati sintetici. Vengono costruite tre serie principali: meteo, consumo e produzione. Il meteo è simulato attraverso temperatura, irradianza e velocità del vento, usando componenti periodiche giornaliere e stagionali con rumore casuale. Il consumo è generato a partire da un profilo giornaliero semplificato, modulato dalla temperatura e da una componente stocastica. La produzione fotovoltaica è invece modellata come funzione dell'irradianza, della capacità installata e di un coefficiente di efficienza costante, con l'aggiunta di rumore. Nel complesso, il notebook costruisce un ambiente sintetico coerente, utile per dimostrare il funzionamento dell'intera pipeline, anche se inevitabilmente semplificato rispetto a un caso reale.

Dopo la generazione dei dati, il flusso passa al preprocessing. In questa fase vengono introdotti valori mancanti in modo artificiale e successivamente imputati tramite interpolazione temporale, seguita da forward fill e backward fill. Questo passaggio simula una pulizia dati realistica e consente di rappresentare almeno in forma minima uno dei problemi tipici delle serie temporali osservate da

sensori o sistemi di monitoraggio. Successivamente, le serie storiche vengono aggregate in un unico dataframe allineato temporalmente.

Il dataset così costruito viene poi suddiviso in tre partizioni: training, validation e test. La suddivisione è cronologica, quindi coerente con la natura del problema di forecasting. Il notebook mantiene anche un buffer temporale nelle partizioni successive per consentire la costruzione delle finestre sequenziali senza perdere continuità. In parallelo viene costruito un dataset sintetico di meteo futuro per i sette giorni successivi, che costituirà l'input esogeno usato nella fase di previsione operativa.

L'approccio di modellazione si basa su due reti neurali sequence-to-sequence separate: una per il consumo e una per la produzione fotovoltaica. In entrambi i casi, l'encoder riceve una sequenza storica composta dalle variabili meteorologiche e dal target passato, mentre il decoder riceve le variabili meteorologiche future. L'uscita del modello è l'intera traiettoria del target sulle 168 ore future. L'architettura scelta è una LSTM encoder-decoder con layer finale `TimeDistributed(Dense(1))`, una soluzione standard per problemi di previsione multi-step con input sequenziali.

Il flusso end-to-end è quindi strutturato nel seguente modo: si generano o si leggono i dati, li si ripulisce, li si divide in training, validation e test, si costruiscono le sequenze per encoder e decoder, si addestrano i due modelli, si valutano le prestazioni, si produce la previsione sui sette giorni successivi e infine si costruisce un report con il bilancio energetico e i grafici di sintesi. L'ultima parte del notebook è dedicata proprio all'output: vengono calcolati il saldo netto, l'energia da acquistare, quella da cedere alla rete e i corrispondenti grafici temporali e cumulativi.

Nel complesso, la struttura del notebook è leggibile e ben organizzata. L'impostazione è chiaramente didattica, ma è sufficientemente completa da rappresentare un prototipo di forecasting energetico con serie temporali multivariate e modello neurale encoder-decoder.

2 Descrizione dell'architettura del modello

Il cuore del notebook è costituito da un'architettura sequence-to-sequence basata su reti LSTM, implementata in due istanze parallele ma concettualmente identiche: una dedicata alla previsione del consumo elettrico e una dedicata alla previsione della produzione fotovoltaica. La logica è quella tipica dei modelli encoder-decoder usati nelle serie temporali multi-step: una prima parte della rete osserva la storia recente del sistema e la comprime in una rappresentazione interna, mentre una seconda parte usa questa rappresentazione insieme alle informazioni future disponibili per generare l'intera traiettoria previsionale.

In termini strutturali, il modello riceve in ingresso due blocchi distinti di dati. Il primo blocco è la sequenza storica dell'encoder, lunga 168 ore, che contiene le variabili meteorologiche osservate nel

passato e il valore storico del target da prevedere, cioè il consumo o la produzione, a seconda del modello considerato. Il secondo blocco è la sequenza futura del decoder, anch'essa lunga 168 ore, che contiene le variabili esogene note o stimate per il futuro, in particolare il meteo previsto sulle ore da forecastare. Questa separazione è molto importante, perché riflette il contesto operativo reale: il modello deve usare il passato per capire lo stato dinamico del sistema, ma per costruire la previsione sulle ore future deve anche appoggiarsi alle informazioni esogene disponibili in anticipo.

L'encoder è composto da un layer LSTM che scorre la sequenza storica passo dopo passo. Una LSTM, cioè Long Short-Term Memory, è una variante delle reti ricorrenti progettata per mantenere memoria delle dipendenze temporali e ridurre i problemi di perdita del gradiente tipici delle RNN tradizionali. Nel notebook questo layer non ha il compito di restituire un output a ogni istante temporale, ma di sintetizzare l'intera finestra passata in uno stato interno finale, formato tipicamente da uno stato nascosto e da uno stato di cella. In termini intuitivi, l'encoder trasforma una settimana di storia in una rappresentazione compatta che contiene informazione su trend recenti, periodicità giornaliera, livello medio del segnale e risposta del target alle condizioni meteorologiche.

Il decoder riceve questa rappresentazione iniziale dell'encoder come condizione di partenza e processa la sequenza delle variabili future. Anche qui il blocco centrale è un layer LSTM, ma con una funzione diversa: invece di comprimere il passato, il decoder espande nel tempo lo stato dinamico appreso e produce una sequenza di stati nascosti, uno per ciascuna delle 168 ore future. In altre parole, il decoder prende memoria del passato e la combina con il meteo futuro per costruire passo dopo passo la previsione dell'intero orizzonte.

Dopo il decoder entra in gioco il layer finale `TimeDistributed(Dense(1))`. Il componente `Dense(1)` è un layer pienamente connesso che trasforma lo stato latente di ciascun time step in un singolo valore numerico, cioè la previsione del target per quell'ora. L'involucro `TimeDistributed` serve ad applicare lo stesso layer dense in modo indipendente a tutti gli istanti della sequenza prodotta dal decoder. Questo significa che la rete restituisce in output una sequenza completa di 168 previsioni orarie, non un solo valore aggregato. È proprio questo aspetto a rendere l'architettura adatta al forecasting multi-step diretto.

Dal punto di vista del flusso informativo, il funzionamento complessivo può essere descritto in tre passaggi. Prima, la storia recente entra nell'encoder e viene condensata in uno stato interno. Secondo, il decoder viene inizializzato con questo stato e riceve in input le variabili meteorologiche future, che forniscono il contesto esogeno ora per ora. Terzo, per ogni time step futuro, il layer `TimeDistributed(Dense(1))` converte la rappresentazione latente in una previsione scalare. Il risultato finale è una curva previsiva completa sulla settimana successiva. Questo schema è particolarmente coerente con il problema studiato, perché consente di separare la memoria del passato dall'informazione futura disponibile, mantenendo al tempo stesso una struttura sufficientemente flessibile da catturare dinamiche non lineari.

Per quanto riguarda i parametri principali, la lunghezza delle sequenze storiche e future è pari a 168 passi temporali, cioè 168 ore, equivalenti a una settimana. Questo iper-parametro definisce quanta memoria il modello osserva e quanto lontano si spinge la previsione. Una finestra più corta riduce la complessità ma rischia di perdere pattern settimanali; una finestra più lunga aumenta il contesto disponibile ma rende l'addestramento più oneroso e può introdurre rumore inutile. La scelta di una settimana è un compromesso ragionevole, perché allinea la memoria storica e l'orizzonte previsionale ai principali cicli giornalieri e settimanali del problema.

Un altro iper-parametro fondamentale è il numero di unità LSTM, cioè la dimensionalità dello stato interno della rete. In pratica questo valore controlla la capacità rappresentativa del modello: un numero più basso rende la rete più semplice e meno incline all'overfitting, ma può limitarne la capacità di apprendere pattern complessi; un numero più alto aumenta la potenza espressiva, ma richiede più dati, più tempo di training e una regolarizzazione più attenta. Nel notebook questo parametro definisce quanta informazione l'encoder può comprimere e quanta ricchezza dinamica il decoder può sviluppare durante la generazione delle previsioni.

La batch size determina invece quante sequenze vengono elaborate insieme a ogni aggiornamento dei pesi. Batch più piccoli rendono l'ottimizzazione più rumorosa ma talvolta più capace di generalizzare; batch più grandi stabilizzano il gradiente e velocizzano il calcolo su hardware parallelo, ma possono richiedere più memoria e portare a minimi meno robusti. Nel contesto del notebook questo parametro incide soprattutto sul compromesso tra stabilità dell'addestramento, tempo computazionale e capacità di generalizzazione.

Il learning rate è il parametro che controlla l'ampiezza degli aggiornamenti durante l'ottimizzazione. Se è troppo alto, il training può diventare instabile e oscillare senza convergere; se è troppo basso, la convergenza diventa lenta e il modello rischia di fermarsi in soluzioni poco buone. In pratica, il learning rate regola la velocità con cui la rete impara. Nel notebook esso lavora insieme all'ottimizzatore per guidare la discesa della loss in modo sufficientemente rapido ma stabile.

La funzione di loss misura l'errore che il modello cerca di minimizzare durante l'addestramento. In problemi di regressione multi-step come questo si usa tipicamente la mean squared error, o una loss equivalente basata sull'errore quadratico medio, perché penalizza maggiormente gli errori grandi e produce gradienti regolari. La scelta della loss influenza il tipo di comportamento che la rete apprende: una loss quadratica spinge il modello a ridurre soprattutto gli scostamenti più marcati, mentre funzioni alternative potrebbero rendere il training più robusto agli outlier ma meno sensibile ai picchi di errore.

L'ottimizzatore, tipicamente Adam in pipeline di questo tipo, è il meccanismo che aggiorna i pesi della rete usando le informazioni del gradiente. Adam combina adattamento individuale del learning rate e memoria dei gradienti passati, risultando spesso una scelta efficace per reti ricorrenti e problemi con molte feature. Il suo ruolo pratico è rendere l'addestramento più stabile e meno

sensibile alla scelta iniziale dei parametri.

L'early stopping è infine un accorgimento di controllo della generalizzazione. Durante il training si monitora la loss di validation e, se questa smette di migliorare per un certo numero di epoche consecutive, l'addestramento viene interrotto automaticamente. Questo meccanismo serve a evitare che la rete continui a migliorare solo sul training set iniziando a peggiorare sui dati non visti. In un notebook come questo l'early stopping ha un'importanza particolare, perché riduce il rischio di overfitting e rende il processo di addestramento più efficiente anche in presenza di dati sintetici e modelli relativamente flessibili.

Nel complesso, l'architettura LSTM encoder-decoder adottata è ben coerente con il problema affrontato. Non è la soluzione più semplice possibile, ma è una scelta sensata quando si vuole prevedere un intero profilo temporale futuro sfruttando sia la memoria storica sia un insieme di covariate esogene disponibili sul futuro. La presenza di due modelli distinti, uno per il consumo e uno per la produzione, consente inoltre di specializzare l'apprendimento su due fenomeni fisicamente diversi, mantenendo però la stessa logica architetturale e lo stesso schema di training.

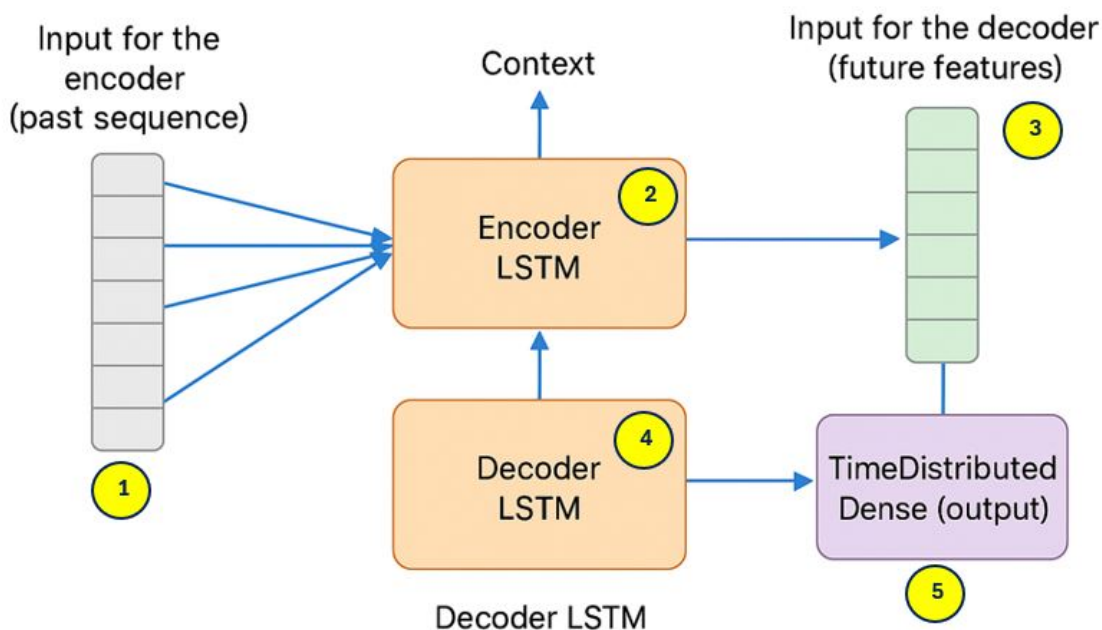


Figura 1: Rappresentazione dell'architettura LSTM encoder-decoder impiegata nel modello di previsione.

La figura sintetizza visivamente il funzionamento della rete in uno stile vicino ai diagrammi usati nella letteratura di machine learning. A sinistra sono rappresentati gli input storici che alimentano l'encoder; al centro compare il passaggio dello stato latente dall'encoder al decoder; a destra si vede la sequenza delle covariate future e il blocco finale che trasforma gli stati del decoder nelle previsioni orarie. Questa rappresentazione è utile anche in ottica editoriale, perché rende immediatamente

leggibile la logica del modello durante l'esportazione del documento in PDF o in Word.

3 Modifiche e miglioramenti introdotti

La modifica metodologica più importante introdotta in questa versione riguarda la gestione dello scaling e, di conseguenza, la valutazione delle prestazioni. Nella versione precedente il punto più critico era la possibile incoerenza tra la scala usata in training e quella usata in validation e test. In pratica, parte dei target era normalizzata, mentre altre porzioni potevano restare in scala originale o essere gestite in modo non completamente coerente durante la costruzione delle sequenze e nella fase di calcolo delle metriche. Questo rendeva le metriche meno affidabili e soprattutto meno interpretabili.

Nella nuova versione il problema è stato affrontato in modo decisamente più rigoroso. È stata introdotta una funzione dedicata, `fit_transform_target_scaler`, che centralizza la gestione della normalizzazione del target. Questa funzione effettua il fit dello scaler esclusivamente sul training set e applica poi la trasformazione a validation e test usando gli stessi parametri. La scelta è corretta dal punto di vista metodologico perché elimina il rischio che informazione statistica proveniente da validation o test entri nel processo di normalizzazione. Il notebook rende anche esplicita la separazione tra target in scala originale e target in scala normalizzata, migliorando leggibilità, tracciabilità e controllo del flusso.

Un secondo miglioramento importante riguarda la fase di valutazione. È stata introdotta la funzione `calcola_metriche_original_scale`, che calcola MAE e RMSE solo dopo aver riportato i valori predetti e quelli reali nella scala fisica originale tramite `inverse transform`. Questo passaggio è fondamentale, perché una metrica calcolata su dati standardizzati non è facilmente interpretabile in termini applicativi. Riportando tutto in kWh, il risultato diventa direttamente leggibile e confrontabile. In questo modo MAE e RMSE risultano finalmente coerenti con il significato operativo del problema.

Il refactoring non si limita al solo calcolo delle metriche, ma coinvolge l'intera coerenza della pipeline. Nella nuova preparazione dei dati, i target di train, validation e test vengono prima separati in forma originale, poi scalati in modo uniforme usando scaler fittati solo sul training, e infine trasformati in sequenze coerenti per tutte le partizioni. Inoltre, il notebook costruisce in parallelo le versioni originali dei target per poter verificare che l'`inverse transform` dei dati scalati restituisca correttamente le serie nella scala fisica. Questo controllo compare in modo esplicito e rappresenta un buon segnale di rigore metodologico.

Anche la parte grafica è stata corretta in linea con questa impostazione. I grafici di confronto tra valori reali e predetti e le distribuzioni degli errori non usano più le serie standardizzate, ma le

versioni in scala originale. Questo è un miglioramento sostanziale, perché evita rappresentazioni grafiche poco leggibili e allinea la visualizzazione con il reporting numerico delle metriche.

Un ulteriore elemento positivo è la presenza di controlli espliciti anti-leakage. Il notebook verifica la coerenza temporale delle partizioni e impone che il fit degli scaler avvenga solo sul training set. Anche se il problema era già riconosciuto nella versione precedente, in questa nuova versione la soluzione è molto più robusta, più leggibile e più facilmente verificabile.

In termini di impatto pratico, la correzione dello scaling cambia in modo rilevante il significato dei risultati. Le prestazioni non sono solo più corrette dal punto di vista numerico, ma soprattutto più affidabili. Un MAE espresso in scala originale può essere confrontato con i valori reali di consumo o produzione e può essere usato per valutare la bontà del modello rispetto a requisiti operativi concreti. In altre parole, il refactoring non è un semplice miglioramento stilistico del codice, ma una correzione metodologica che aumenta la validità dell'intero processo di valutazione.

Questa nuova versione del notebook rappresenta quindi un passo avanti significativo rispetto alla precedente. Non modifica l'impostazione generale del modello, ma corregge uno degli aspetti più delicati di qualsiasi pipeline di machine learning su serie temporali: la coerenza tra preprocessing, addestramento e valutazione.

4 Sintesi valutazione modello (post-refactoring scaling)

Alla luce del refactoring introdotto nella gestione dello scaling, la valutazione dei modelli assume ora un significato molto più solido dal punto di vista metodologico. Le metriche non sono più calcolate su serie standardizzate o potenzialmente incoerenti tra training, validation e test, ma vengono riportate nella scala fisica originale prima del calcolo finale. Questo rende i risultati finalmente interpretabili e confrontabili in termini reali, e permette di leggere le prestazioni del modello in modo corretto anche dal punto di vista applicativo.

Nel caso del modello di consumo, le prestazioni risultano molto convincenti lungo tutte le partizioni del dataset. Sul training set il modello raggiunge un MAE pari a 0.1670 e un RMSE pari a 0.2091, valori che indicano un fit molto buono e non evidenziano segnali di apprendimento instabile. Sulla validation il MAE sale in modo contenuto a 0.1911 e il RMSE a 0.2387, mostrando una buona capacità di generalizzazione. Anche sul test i risultati restano stabili, con MAE pari a 0.1864 e RMSE pari a 0.2347. Nel complesso, il comportamento del modello di consumo appare coerente tra training, validation e test, senza segnali evidenti di overfitting. Il miglioramento numerico rispetto alla versione precedente è molto elevato, dell'ordine di circa il -90% sul MAE e di circa il -88% sul RMSE, ma tale variazione non va letta come un puro salto prestazionale del modello: essa dipende in larga misura dalla correzione della pipeline di scaling e dal fatto che ora le metriche

sono finalmente calcolate nella scala giusta.

Anche il modello di produzione fotovoltaica mostra un comportamento molto regolare. Sul training set ottiene un MAE di 0.0765 e un RMSE di 0.0958, confermando un fit molto buono. Sulla validation i valori restano molto vicini, con MAE pari a 0.0826 e RMSE pari a 0.1029, mentre sul test si osservano MAE pari a 0.0796 e RMSE pari a 0.0995. Anche in questo caso la distanza tra training, validation e test è ridotta, e questo suggerisce una buona generalizzazione del modello e l'assenza di overfitting marcato. Il miglioramento numerico rispetto alla configurazione precedente è anch'esso molto rilevante, pari a circa il -85% sul MAE e il -86% sul RMSE. Tuttavia, come per il modello di consumo, il significato principale di questo miglioramento non sta tanto in una rivoluzione architetturale, quanto nel fatto che la valutazione è ora metodologicamente corretta e i risultati sono finalmente omogenei tra le diverse partizioni del dataset.

L'interpretazione complessiva di questa nuova fase di valutazione è quindi positiva. Le metriche sono adesso calcolate in scala originale, i risultati sono coerenti e confrontabili, e il comportamento dei modelli su training, validation e test suggerisce una generalizzazione buona, senza evidenze immediate di sovra-adattamento. Allo stesso tempo, è importante sottolineare che l'entità del miglioramento numerico osservato è dovuta in misura significativa alla correzione dello scaling e non può essere interpretata come prova definitiva di superiorità del modello in senso assoluto. Per questo motivo, i risultati devono essere letti come un consolidamento metodologico della pipeline più che come una validazione conclusiva del sistema.

In prospettiva, i prossimi miglioramenti più importanti riguardano l'introduzione di benchmark espliciti, come persistence e seasonal naive, l'uso di dati reali al posto dei dati sintetici, l'arricchimento dello spazio delle feature e una valutazione più realistica dell'impatto dell'errore meteorologico sulle previsioni finali. Questi passaggi sono essenziali per trasformare l'attuale buon risultato interno del notebook in una valutazione più credibile e trasferibile a un contesto operativo reale.

5 Criticità residue e possibili evoluzioni

Nonostante il miglioramento introdotto sul fronte dello scaling, il notebook presenta ancora diverse criticità metodologiche e applicative che ne limitano la robustezza come strumento di previsione reale.

La prima criticità rimasta è la forte dipendenza da dati sintetici. Anche con una pipeline di scaling corretta, il notebook continua a essere addestrato e valutato su serie generate artificialmente. Questo significa che il modello apprende pattern prodotti dalle regole del simulatore, non necessariamente pattern realistici osservati nel comportamento energetico di un utente reale. Il rischio è che le metriche risultino ottimistiche perché il problema è costruito in un ambiente controllato

e semplificato. La soluzione concreta è passare a una pipeline real-data-first, in cui il notebook carichi come input dati storici reali di consumo, produzione e meteo. Dal punto di vista tecnico, questo richiede una fase di ingestione dati con lettura da CSV o database, normalizzazione delle unità di misura, verifica della continuità temporale e allineamento di tutte le serie su un indice orario comune.

Una seconda criticità è la semplicità del modello di consumo. Il consumo sintetico dipende essenzialmente da ora del giorno, temperatura e rumore. Mancano fattori importanti come differenza tra giorni feriali e festivi, stagionalità comportamentali, occupancy pattern, utilizzo di elettrodomestici, effetto delle tariffe e possibili anomalie. Di conseguenza il problema resta troppo regolare e semplificato. La soluzione tecnica consiste nell'introdurre feature engineered più ricche, come codifiche cicliche di ora e giorno della settimana, indicatori di weekend e festività, variabili lag del target, medie mobili e aggregati temporali. In una versione più avanzata, si potrebbe anche prevedere un modulo di feature engineering esplicito che arricchisca il dataframe prima della costruzione delle sequenze.

Una terza criticità riguarda la produzione fotovoltaica. Il modello attuale usa irradianza, capacità ed efficienza costante, ma trascura temperatura del modulo, nuvolosità reale, orientamento e inclinazione dei pannelli, perdite e clipping dell'inverter. Questo produce una serie troppo deterministica. La soluzione più solida è utilizzare dati reali di produzione. In alternativa, se si vuole mantenere la simulazione, occorrerebbe introdurre un modello fisicamente più ricco, con derating termico, attenuazione dovuta alle condizioni meteo e saturazione alla potenza massima dell'impianto.

Un'altra debolezza ancora presente riguarda l'uso del meteo futuro. Il notebook usa variabili meteorologiche future come input del decoder, ma le tratta di fatto come perfettamente note. In un contesto reale, invece, anche il meteo è oggetto di previsione e quindi contiene errore. Questo può rendere i risultati del modello più ottimistici di quanto sarebbero in esercizio. Una soluzione concreta consiste nel distinguere tra meteo osservato e forecast meteorologico, introducendo un dataset di input futuro proveniente da un modello meteorologico o da una fonte esterna. Dal punto di vista tecnico, si potrebbe anche lavorare con scenari multipli o intervalli di incertezza, valutando la sensibilità del forecast energetico a errori nelle variabili esogene.

Permane inoltre una criticità nella valutazione comparativa. Il notebook, pur avendo corretto le metriche, continua a non confrontare i modelli LSTM con baseline più semplici. Senza benchmark come persistenza, seasonal naive, regressione lineare o modelli ad albero, non è possibile stabilire se la complessità della rete neurale sia realmente giustificata. La soluzione è introdurre una sezione di benchmarking con modelli semplici e una tabella finale di confronto sulle stesse partizioni e nelle stesse unità fisiche. Questo renderebbe molto più solido il giudizio sulla reale utilità del modello.

Un altro punto debole è la parziale eterogeneità del notebook dal punto di vista editoriale e strutturale. Si notano blocchi descrittivi aggiunti successivamente, celle con marker di confronto tra

vecchio e nuovo codice e una struttura che resta in parte ibrida tra notebook operativo e notebook di audit. Questo non compromette la logica, ma riduce la pulizia del prodotto finale. La soluzione tecnica è produrre una versione production-ready del notebook, ripulita dai blocchi di confronto, con sezioni più nette e funzioni raccolte in modo modulare.

Infine, rimane una limitazione pratica rilevante: il notebook dipende da TensorFlow/Keras e da una configurazione locale che potrebbe non essere sempre riproducibile in tutti gli ambienti. La soluzione è aggiungere una sezione iniziale di controllo ambiente o, meglio ancora, esportare la logica in uno script Python modulare o in una pipeline più robusta, lasciando al notebook solo il ruolo di report esplorativo o dimostrativo.

In sintesi, la versione aggiornata rappresenta un miglioramento metodologico concreto e importante, perché corregge correttamente la parte di normalizzazione e rende finalmente affidabili e interpretabili le metriche. Tuttavia, il notebook resta un prototipo avanzato più che un sistema maturo. Le evoluzioni più importanti da implementare riguardano il passaggio a dati reali, il miglioramento della rappresentazione delle variabili, l'introduzione di benchmark comparativi e una gestione più realistica dell'incertezza sul meteo futuro. Se questi interventi verranno realizzati, il notebook potrà evolvere da dimostratore ben costruito a strumento predittivo molto più credibile e robusto.

6 Conclusioni

Il percorso di revisione descritto in questo documento ha permesso di trasformare un prototipo inizialmente utile soprattutto in chiave esplorativa in una versione molto più solida dal punto di vista metodologico. L'analisi condotta con Prism ha avuto un ruolo centrale in questo processo, perché ha consentito di esaminare con maggiore profondità la struttura della pipeline, di individuare le principali criticità residue e di ricondurre l'intero flusso di lavoro a criteri più rigorosi di coerenza tra preprocessing, addestramento e valutazione. In particolare, il nodo più delicato è stato identificato nella gestione dello scaling del target, che nella configurazione precedente rendeva ambigua l'interpretazione delle metriche e indeboliva la leggibilità complessiva dei risultati.

L'intervento di revisione non si è limitato a una correzione locale, ma ha portato a un vero refactoring della pipeline. La separazione tra training, validation e test è stata resa più chiara e metodologicamente corretta, il fit degli scaler è stato ricondotto esclusivamente ai dati di training e la valutazione finale è stata riportata nella scala fisica originale delle grandezze previste. Questo ha permesso di ottenere metriche finalmente interpretabili e affidabili, non solo perché numericamente più convincenti, ma soprattutto perché coerenti con il significato operativo del problema. In tal modo, il miglioramento ottenuto non va letto come un semplice incremento delle performance del modello, bensì come un rafforzamento sostanziale della sua validità metodologica.

Proprio questo è forse il risultato più importante dell'intero lavoro: il modello risulta oggi più credibile, più trasparente e più adatto a essere discusso in prospettiva applicativa reale. La maggiore coerenza tra le diverse partizioni del dataset, la riduzione delle ambiguità nel preprocessing e la corretta interpretazione delle metriche rendono infatti il sistema una base più robusta per ulteriori sviluppi. Resta naturalmente il carattere prototipale del notebook, soprattutto per la dipendenza da dati sintetici e per l'assenza di benchmark esterni più semplici, ma la revisione effettuata ha costruito un fondamento molto più affidabile su cui proseguire.

In prospettiva, le evoluzioni più interessanti riguardano l'impiego di dati reali, l'introduzione di baseline comparative, l'arricchimento delle feature e una gestione più realistica dell'incertezza meteorologica futura. Su queste basi, il lavoro svolto fin qui non rappresenta soltanto una correzione del modello esistente, ma l'avvio di una traiettoria di maturazione che può portare il sistema verso applicazioni predittive progressivamente più credibili, robuste e utili in contesti operativi concreti.